

SOFTWARE ENGINEERING - LEVEL 3 MINI-PROJECT

Software Requirements Specification

Simple Shell - Command Line Interpreter

Item	Details
Project Title	Simple Shell - Command Line Interpreter
Group	Team 3
Course	UE24CS341A - Software Engineering
Technology	C11 / POSIX / Linux
Methodology	Agile
Team	Rekha Dhorigol (PES1UG24CS370) Rohan Rampura Praveen Kumar (PES1UG24CS382) Samika Ojha (PES1UG24CS412) Sanjana Subrahmanya Aithal (PES1UG24CS421)
Version	1.1 - Revised Requirements Baseline

Revision History

Version	Date	Description
1.0	2026-09-07	Initial requirements baseline

1. Introduction

1.1 Purpose

This document specifies the functional, non-functional and security requirements for Team 3's Simple Shell project. The shell is implemented in C for a Linux/POSIX environment and is developed as a modular Software Engineering mini-project. This version is the requirements baseline for Deliverable 2.

1.2 Intended Audience

The document is intended for the project team, testers, faculty and evaluators. It defines what the system shall do and provides the basis for later design, implementation, testing and validation.

1.3 Product Scope

The Simple Shell is a lightweight command-line interpreter. It reads a command, parses it, executes supported built-ins or external programs, and provides foreground/background execution, redirection, pipelines, current-session history and environment-variable operations.

1.4 References

- SE Level 3 Mini-Project — Student Submission Guide & Complete Exemplar (Coffee Kiosk), PES University
- Jackfruit — Software Engineering Project Guidelines 2026, PES University
- POSIX.1-2017, relevant process, signal and I/O interfaces
- Team 3 Simple Shell Synopsis, Deliverable 1

2. Overall Description

2.1 Product Perspective

The shell is a standalone terminal application. It consists of four functional areas: F1 Command Interpretation & Built-ins, F2 Process Execution & Job Control, F3 I/O Redirection & Pipelines, and F4 Command History & Shell Environment. The modules use standard C/POSIX facilities and do not require a database, server or network service.

2.2 Product Functions

- Display a prompt and read one command line.
- Tokenize and parse the command and arguments.
- Execute the built-ins cd, pwd, echo, exit and help.
- Execute external programs using POSIX processes.
- Support foreground and background execution using '&'.
- Support input (<), output (>) and append (>>) redirection.
- Support pipelines containing up to five commands.
- Store and display commands entered during the current session.
- Support export, unset and env for current-session environment variables.
- Report invalid input and execution errors without crashing.

2.3 User Classes

User Class	Description
Students / Learners	Use the shell to understand basic OS and process concepts.
Command-line Users	Use the shell interactively with standard user privileges.
Developers / Testers	Build, inspect and test the source code and modules.
Faculty / Evaluators	Assess functional correctness, SDLC evidence and individual contribution.

2.4 Scope

In Scope	Out of Scope
Interactive prompt and input	GUI
Parsing and command execution	Remote/network execution
cd, pwd, echo, exit, help	Full shell scripting language
Foreground/background execution	Root/admin management
<, > and >>	Persistent cross-session history
Pipelines up to 5 commands	Loops, conditionals and shell functions
Current-session history	Command recall/editing
export, unset and env	Database/network services

2.5 Operating Environment

- Linux/POSIX-compatible operating system; Ubuntu/Linux used for development and testing.
- C11 compiled with GCC.
- Interactive terminal (TTY); no GUI.

2.6 Constraints

- Use POSIX APIs such as fork(), exec-family calls, wait/waitpid(), pipe(), dup2() and signal handling.
- Core shell parsing shall be implemented by the team.
- Use Git/GitHub and the agreed Agile workflow.
- Dynamic memory and file descriptors shall be released appropriately.
- The four functional areas shall remain modular and testable.

2.7 Assumptions and Dependencies

- Single-user Linux environment with standard POSIX libraries.
- External commands exist on PATH or are supplied using a valid executable path.
- No third-party runtime service is required.

3. External Interface Requirements

3.1 User Interface

- A text prompt such as 'shell> ' shall be displayed.
- The user enters commands through standard keyboard input.
- Output and errors use standard output/error streams.
- Invalid input produces an error and returns to the prompt.
- The history built-in lists current-session commands with numeric indices.

3.2 Software Interfaces

Interface	Purpose
fork(), execvp()/execve(), waitpid()	Create, execute and wait for processes.
open(), close(), dup2(), pipe()	Implement redirection and pipelines.
signal()/sigaction()	Handle interrupts and child-process signals.
C standard library	String, memory and argument handling.
getenv(), setenv(), unsetenv(), environ	Environment-variable operations.

3.3 Communications Interfaces

Not applicable. The shell is a local application and has no network communication requirement.

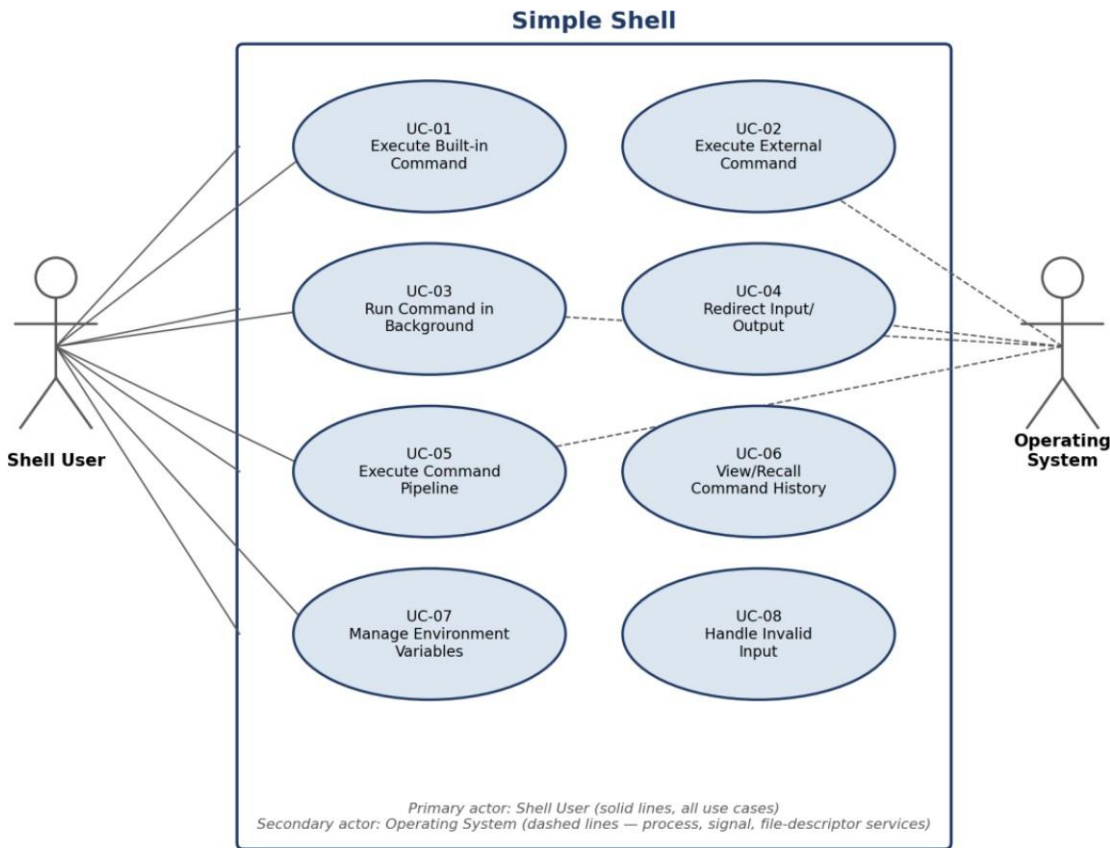
4. Analysis Models

4.1 Actors

The primary actor is the Shell User. The Operating System provides process, file and signal services.

4.2 Use Case Summary

ID	Use Case	Outcome
UC-01	Execute Built-in Command	Supported built-in executes; prompt returns.
UC-02	Execute External Command	External program executes; shell resumes.
UC-03	Run Command in Background	Shell returns to prompt without waiting.
UC-04	Redirect Input/Output	stdin/stdout is connected to a specified file.
UC-05	Execute Command Pipeline	Two to five commands are connected by pipes.
UC-06	View Command History	Current-session commands are displayed with indices.
UC-07	Manage Environment Variables	Variable is set, removed or listed.
UC-08	Handle Invalid Input	Error is shown and shell continues.



4.3 Use Case Diagram

The requirements-level diagram shall contain the Shell User and Operating System actors and the eight use cases above. Detailed sequence diagrams are deferred to Deliverable 3 (Design).

5. System Features

Ownership rule: each feature has one primary owner for end-to-end functionality, but all four team members contribute to requirements, design, implementation, testing, review and validation.

Feature	Primary Owner	Scope
F1 Command Interpretation & Core Built-in Commands	Rekha Dhorigol	Parsing, classification, core built-ins.
F2 Process Execution & Job Control	Rohan Rampura Praveen Kumar	External commands, foreground/background execution and basic signal handling.
F3 I/O Redirection & Pipelines	Samika Ojha	<, >, >> and pipelines.
F4 Command History & Shell Environment	Sanjana Subrahmanya Aithal	Current-session history and environment operations.

5.1 Command Interpretation & Core Built-ins (F1)

Owner: Rekha Dhorigol

ID	Requirement	Priority	Verification
FR-CI-01	The shell shall display a prompt and accept one input line of at most 1024 characters.	High	Functional test
FR-CI-02	The shell shall tokenize the input into a command and zero or more arguments, with at most 64 arguments.	High	Unit test
FR-CI-03	The shell shall classify a parsed command as a supported built-in or external command.	High	Unit test
FR-CI-04	The shell shall implement cd, pwd, echo, exit and help.	High	Functional test
FR-CI-05	The shell shall display an error for empty, malformed or unrecognised input and return to the prompt.	High	Negative test
FR-CI-06	The shell shall reject input longer than 1024 characters without crashing or corrupting shell state.	High	Boundary test
FR-CI-07	The exit built-in shall terminate the shell cleanly and release allocated resources.	Medium	Functional + memory test

5.2 Process Execution & Job Control (F2)

Owner: Rohan Rampura Praveen Kumar

ID	Requirement	Priority	Verification
FR-PE-01	The shell shall create a child process using fork() for each external command.	High	Unit test
FR-PE-02	The child shall execute the requested program using an exec-family call and the parsed arguments.	High	Functional test
FR-PE-03	For foreground execution, the shell shall wait for the child before displaying the next prompt.	High	Functional test
FR-PE-04	For a command ending with &, the shell shall start the command and return to the prompt without waiting.	High	Functional test
FR-PE-05	The shell shall report when a foreground command terminates abnormally.	Medium	Functional test
FR-PE-06	The shell shall reap terminated background children to prevent zombie processes.	High	Integration test
FR-PE-07	The shell shall handle Ctrl+C so that the shell itself is not terminated by an interrupt intended for a foreground command.	High	Signal test

5.3 I/O Redirection & Pipelines (F3)

Owner: Samika Ojha

ID	Requirement	Priority	Verification
FR-IO-01	The shell shall redirect stdout using > and replace existing file contents.	High	Functional test
FR-IO-02	The shell shall redirect stdout using >> and append to an existing file.	High	Functional test
FR-IO-03	The shell shall redirect stdin using < from a specified file.	High	Functional test
FR-IO-04	The shell shall support a two-command pipeline using .	High	Integration test
FR-IO-05	The shell shall support pipelines containing up to five commands.	Medium	Integration test
FR-IO-06	The shell shall display an error and skip execution when a required redirection file cannot be opened.	High	Negative test
FR-IO-07	The shell shall close unused pipe descriptors to avoid descriptor leaks and hangs.	High	Resource test

5.4 Command History & Shell Environment (F4)

Owner: Sanjana Subrahmanya Aithal

ID	Requirement	Priority	Verification
FR-HE-01	The shell shall store successfully parsed command lines in an in-memory history buffer of at most 50 entries.	Medium	Unit test
FR-HE-02	The history built-in shall display stored commands in order with numeric indices.	Medium	Functional test
FR-HE-03	When 50 entries are stored, adding a new entry shall remove the oldest entry.	Medium	Unit test
FR-HE-04	The export built-in shall set or update an environment variable for the current session.	High	Functional test
FR-HE-05	The unset built-in shall remove an environment variable from the current session.	Medium	Functional test
FR-HE-06	The env built-in shall list environment variables currently available to the shell.	Medium	Functional test
FR-HE-07	Environment-variable changes shall be visible to subsequently created child processes.	High	Integration test

6. Non-Functional Requirements

6.1 Performance

ID	Requirement	Verification
NFR-01	The shell shall redisplay the prompt within 1 second after a normal foreground command completes, under the reference test environment.	Performance test
NFR-02	The shell shall be ready to accept input within 1 second of launch under the reference test environment.	Performance test
NFR-03	A pipeline of up to five lightweight test commands shall execute without deadlock or indefinite blocking.	Integration test

6.2 Safety

- The shell shall not provide hidden destructive operations.
- Terminated child processes shall be reaped appropriately.

6.3 Security

ID	Requirement	Validation
----	-------------	------------

SEC-01	The shell shall not use <code>system()</code> , <code>popen()</code> or equivalent shell-interpolation functions to execute user commands; commands shall be passed as argument vectors to <code>exec</code> .	Code review + negative test
SEC-02	The shell shall reject input longer than 1024 characters and shall not copy it beyond allocated bounds.	Boundary test + static analysis
SEC-03	The shell shall not print or log environment-variable values unless the user explicitly requests them through <code>env</code> .	Code review
SEC-04	The shell shall handle child-process signals without terminating unexpectedly and shall safely reap terminated children.	Signal/negative test
SEC-05	Redirection shall occur only when the user explicitly supplies <code><</code> , <code>></code> or <code>>></code> and shall use the invoking user's OS permissions.	Functional + negative test
SEC-06	Parsing and history memory shall be released exactly once and checked for leaks or invalid access.	Valgrind/ASan + code review

6.4 Quality Attributes

- Correctness: defined commands shall produce expected results in the test cases.
- Reliability: invalid input shall not crash the shell.
- Maintainability: the four functional areas shall remain modular.
- Testability: each functional area shall have focused unit/integration tests.

6.5 Domain Rules

- The shell operates under the invoking user's OS permissions.
- History is current-session only and limited to 50 entries.
- A pipeline may contain at most five commands.
- Only features explicitly listed in this SRS are required.

7. Other Requirements

7.1 Memory and Resource Management

- Dynamic memory used for parsing and history shall be released appropriately.
- File descriptors created for redirection and pipelines shall be closed when no longer required.
- Child processes shall be waited for or reaped as appropriate.

7.2 Fixed Limits

Item	Limit
Maximum command-line length	1024 characters
Maximum arguments per command	64
Maximum history entries per session	50
Maximum pipeline stages	5 commands

7.3 Portability

The project targets Linux/POSIX systems. Portability to non-POSIX operating systems is not required.

7.4 External Dependencies

No database, cloud service or network service is required at runtime. The shell depends on the Linux/POSIX environment and standard C/POSIX libraries.

8. Initial Validation Specification

Validation will be performed as features are implemented. The verification method for every functional, security and non-functional requirement is stated in Sections 5 and 6.

Validation Areas

Area	Checks
------	--------

Parsing / built-ins	Prompt, tokenization, built-ins, invalid input and 1024-character boundary.
Processes	Foreground/background execution, abnormal termination, child reaping and Ctrl+C.
Redirection	<, > and >> with valid and inaccessible files.
Pipelines	Two-stage and up to five-stage pipelines; no deadlock; descriptors closed.
History	Recording, ordered display, 50-entry boundary and oldest-entry removal.
Environment	export, unset, env and visibility in child processes.
Security / robustness	No shell interpolation, bounded input, signal handling and memory checks.
Performance	Startup and prompt-response measurements under the same reference environment.

Reference Test Environment

Ubuntu/Linux system, GCC and a standard terminal session. Measurements shall be recorded on the same reference machine when possible.

9. Project Plan

9.1 Work Breakdown Structure

WP	Work Package	Coordination
WP1	Requirements baseline	All members
WP2	Architecture and design	All members
WP3	F1 Command Interpretation	Rekha
WP4	F2 Process Execution	Rohan
WP5	F3 Redirection & Pipelines	Samika
WP6	F4 History & Environment	Sanjana
WP7	Integration, CI and security checks	All members
WP8	Final validation, report and demo	All members

9.2 Six-Sprint Plan

Work Package	S1	S2	S3	S4	S5	S6
WP1 Requirements	X					
WP2 Architecture/Design	X	X				
WP3 F1		X				
WP4 F2			X			
WP5 F3				X		
WP6 F4					X	
WP7 Integration/CI/Security					X	
WP8 Validation/Report/Demo						X

9.3 Gantt View

WP	S1	S2	S3	S4	S5	S6
Requirements	■					
Architecture	■	■				
F1		■				
F2			■			
F3				■		
F4					■	
Integration/CI/Security					■	
Validation/Report/Demo						■

10. Tool Stack

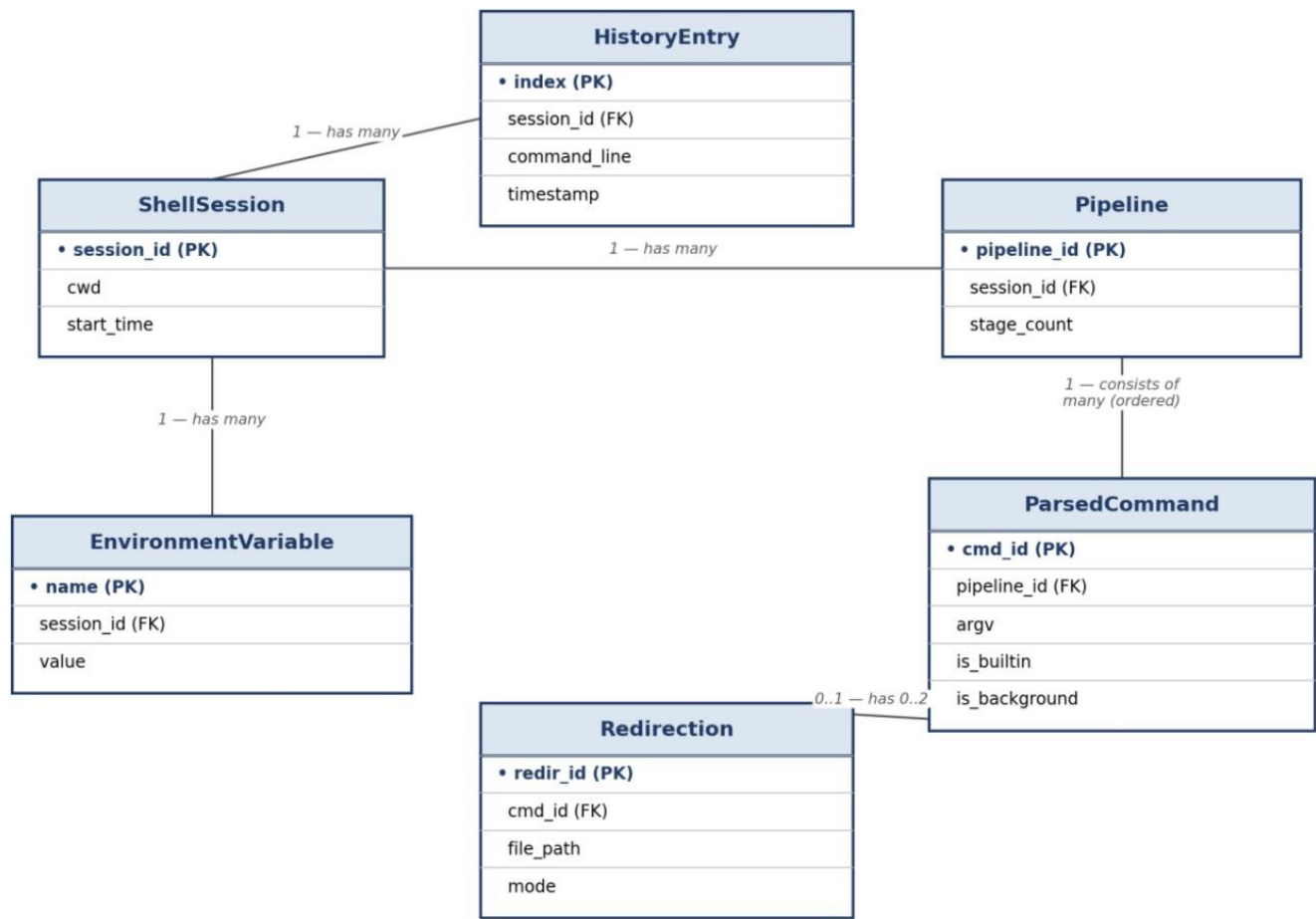
Activity	Tool	Evidence
Version control	Git + GitHub	Repository and commit history
Agile planning	Jira or GitHub Projects	Backlog and sprint board
Build	GCC + Make	Makefile and build output
CI/CD	Jenkins	Pipeline log
Containerization	Docker	Dockerfile and build/run evidence
Testing	Check / CMocka / custom C test harness	Test results
Static analysis	cppcheck or suitable C tool	Analysis report
Memory/robustness	Valgrind or AddressSanitizer	Memory/leak report
Design	Draw.io / StarUML / PlantUML	UML diagrams

Appendix A: Glossary

Term	Definition
Built-in command	A command implemented directly within the shell process, such as cd or exit.
External command	A program executed by creating a child process and using an exec-family call.
Foreground execution	Execution in which the shell waits for the command to finish.
Background execution	Execution in which the shell starts the command and returns to the prompt without waiting.
Redirection	Connecting standard input/output to a file using <, > or >>.
Pipeline	Two to five commands connected so stdout of one becomes stdin of the next.
Job control	Basic management of foreground/background processes and related signals.
Zombie process	A terminated child whose exit status has not yet been collected by its parent.
Environment variable	A named value in the process environment that may be inherited by child processes.

Appendix B: Field Layouts

The shell has no external database. These are the key in-memory structures required by the current requirements.



In-memory data model — the Simple Shell has no persistent database; these entities and relationships exist only within a single running shell session (see Appendix B for field layouts).

B.1 History Entry

Field	Type	Description	Mandatory
index	int	Position in the current-session history buffer.	Y
command_line	char[]	Stored command line, limited to 1024 characters.	Y

B.2 Parsed Command

Field	Type	Description	Mandatory
argv	char*[]	NULL-terminated argument vector; maximum 64 arguments.	Y
is_builtin	bool	Whether the command is a supported built-in.	Y
input_file	char*	Input redirection path, if present.	N
output_file	char*	Output redirection path, if present.	N
append_mode	bool	True when >> is used.	N
is_background	bool	True when the command uses background execution.	Y

B.3 Pipeline

Field	Type	Description	Mandatory
commands	ParsedCommand[]	Commands in the pipeline.	Y
stage_count	int	Number of commands; maximum 5.	Y

B.4 Environment Variable Entry

Field	Type	Description	Mandatory
name	char*	Environment variable name.	Y
value	char*	Environment variable value.	Y

Appendix C: Requirement Traceability Matrix

At the Requirements stage, architecture, design, code and test references remain TBD. They will be completed in later deliverables.

Req ID	Brief Description	Arch.	Design	Code	Test
FR-CI-01	Prompt; input ≤1024 chars	TBD	TBD	TBD	TBD
FR-CI-02	Tokenize; max 64 args	TBD	TBD	TBD	TBD
FR-CI-03	Classify built-in/external	TBD	TBD	TBD	TBD
FR-CI-04	Core built-ins	TBD	TBD	TBD	TBD
FR-CI-05	Error handling	TBD	TBD	TBD	TBD
FR-CI-06	Reject >1024 chars	TBD	TBD	TBD	TBD
FR-CI-07	Clean exit	TBD	TBD	TBD	TBD
FR-PE-01	fork()	TBD	TBD	TBD	TBD
FR-PE-02	exec-family	TBD	TBD	TBD	TBD
FR-PE-03	Foreground wait	TBD	TBD	TBD	TBD
FR-PE-04	Background execution	TBD	TBD	TBD	TBD
FR-PE-05	Abnormal termination	TBD	TBD	TBD	TBD
FR-PE-06	Reap children	TBD	TBD	TBD	TBD
FR-PE-07	Ctrl+C handling	TBD	TBD	TBD	TBD
FR-IO-01	Output redirection >	TBD	TBD	TBD	TBD
FR-IO-02	Append redirection >>	TBD	TBD	TBD	TBD
FR-IO-03	Input redirection <	TBD	TBD	TBD	TBD
FR-IO-04	Two-command pipeline	TBD	TBD	TBD	TBD
FR-IO-05	Pipeline max 5 stages	TBD	TBD	TBD	TBD
FR-IO-06	Redirection error	TBD	TBD	TBD	TBD
FR-IO-07	Close unused descriptors	TBD	TBD	TBD	TBD
FR-HE-01	History max 50 entries	TBD	TBD	TBD	TBD
FR-HE-02	Display history	TBD	TBD	TBD	TBD
FR-HE-03	Remove oldest when full	TBD	TBD	TBD	TBD
FR-HE-04	export	TBD	TBD	TBD	TBD
FR-HE-05	unset	TBD	TBD	TBD	TBD
FR-HE-06	env	TBD	TBD	TBD	TBD
FR-HE-07	Environment reaches children	TBD	TBD	TBD	TBD
SEC-01	No shell interpolation	TBD	TBD	TBD	TBD
SEC-02	Bounded input	TBD	TBD	TBD	TBD
SEC-03	No unintended env exposure	TBD	TBD	TBD	TBD
SEC-04	Safe child signal handling	TBD	TBD	TBD	TBD
SEC-05	Explicit redirection	TBD	TBD	TBD	TBD
SEC-06	Safe memory handling	TBD	TBD	TBD	TBD
NFR-01	Prompt response ≤1s	TBD	TBD	TBD	TBD
NFR-02	Startup ≤1s	TBD	TBD	TBD	TBD
NFR-03	Pipeline up to 5 stages; no deadlock	TBD	TBD	TBD	TBD